

Preliminary Conceptual Model

Ontology-Guided Agentic Capabilities Orchestration (AC-001)

Team AgenticCapabilities · Ontology Engineering 2026, Assignment 5 · Version 1.1, October 1, 2026

Gia Oriana Santos, Barbara Kotlan, Jiaying (Michael) Wang, Jialin (Gary) Fang, Elíoenai Lincon Guedes Alves

This model covers the orchestration layer of use case AC-001: what a user request asks for, which capabilities, skills and tools can meet it, how candidate orchestration plans are built and compared, and what the knowledge graph (KG) records when a plan runs. The supplement domain enters only through the kinds of information artifacts that skills exchange (Figure 8). The model has 64 classes and 34 object properties in eight UML class diagrams: Figure 1 connects the core classes, and Figures 2–8 each detail one area. The deepest is-a chain has four levels (Constraint, ExecutionConstraint, Precondition, InputConstraint).

Notation. UML 2.5.1 class diagrams drawn with PlantUML. A box is a class; attributes inside a box are datatype properties. A line with a hollow triangle points from a subclass to its superclass (is a). A labeled arrow is an object property drawn from its domain to its range; a dashed arrow whose name starts with “/” is derived, meaning the reasoner infers it. «AgentO» marks classes reused from AgentO [2], [3]; «defined» marks classes whose members the reasoner infers from a definition. A gray box is a class detailed in another figure. Property prefixes name reused vocabularies: agento: AgentO [3], prov: PROV-O [9], process: OWL-S [8], pplan: P-Plan [12]; properties without a prefix are new in this model.

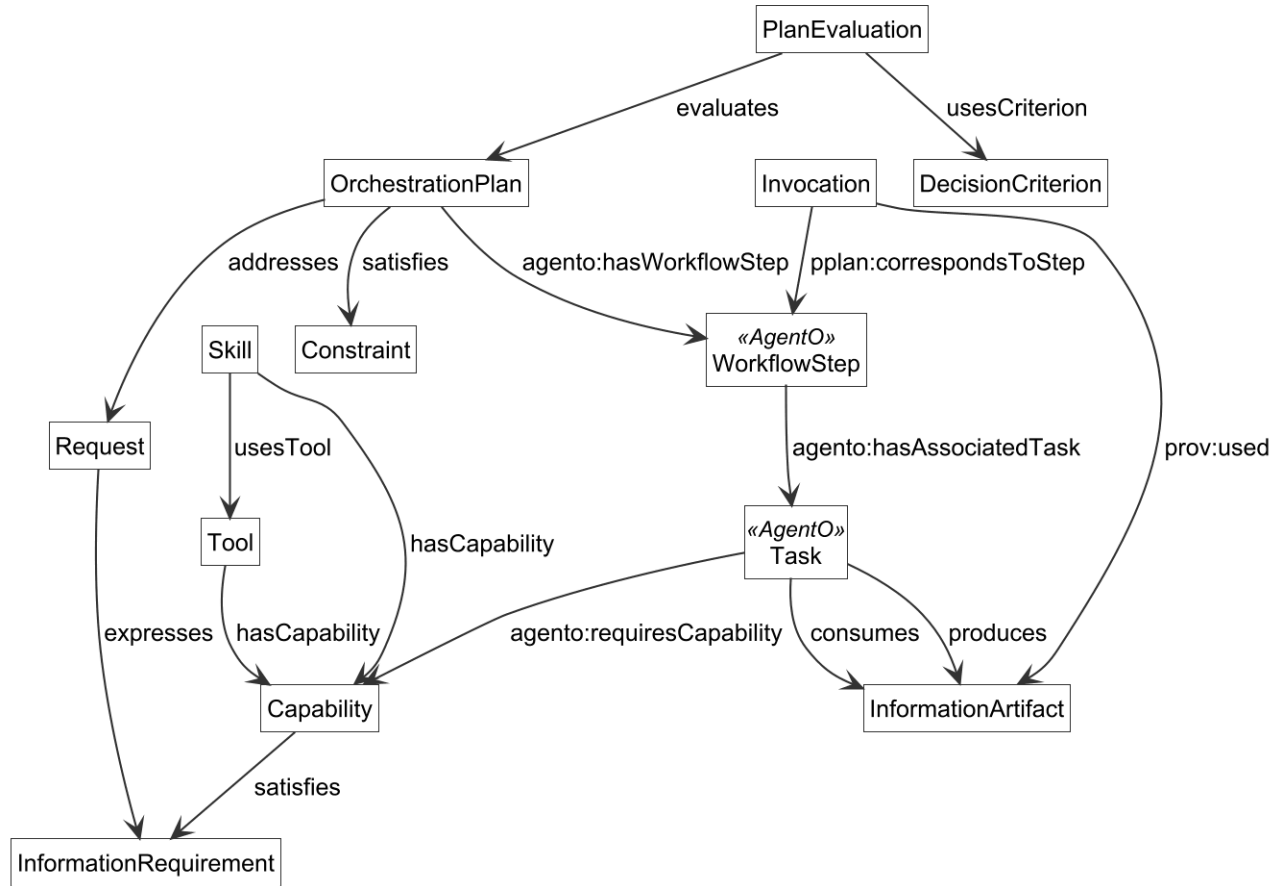


Figure 1. Overview of the core classes

Shows. The core class of each area and the relationships that tie the areas together. A Request expresses InformationRequirements, which Capabilities satisfy; Skills and Tools have Capabilities. An OrchestrationPlan addresses the Request and satisfies Constraints; its WorkflowSteps are associated with Tasks, and each Task requires a Capability and consumes and produces InformationArtifacts. PlanEvaluations compare plans on DecisionCriteria. When the plan runs, each Invocation corresponds to one WorkflowStep and uses InformationArtifacts. To keep the overview small, evaluations and constraint satisfaction attach here to OrchestrationPlan; Figure 6 places them on candidate plans.

Competency questions. The path Request → InformationRequirement ← Capability ← Skill answers CQ 3; OrchestrationPlan → WorkflowStep → Task → Capability answers CQ 5.

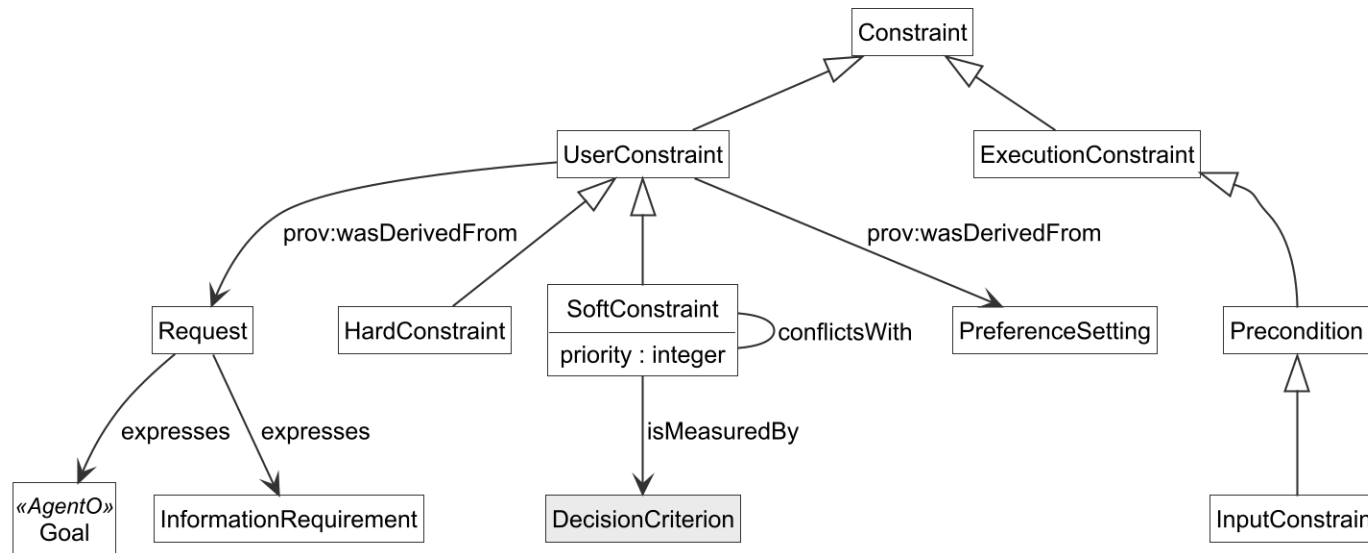


Figure 2. Requests, requirements and constraints

Shows. What the NLP tool and harness extract in basic flow steps 2–3: the Goals and InformationRequirements a Request expresses, and the UserConstraints that steer plan selection. `prov:wasDerivedFrom` records whether a constraint came from the Request or from the user’s PreferenceSettings. PreferenceSettings are what a user saves before making requests; no step consumes or produces them, so they are not information artifacts and stay outside the artifact hierarchy of Figure 8.

Modeling decisions. User constraints are split by how they are used. A HardConstraint must hold, for example Jessica’s safety screening, which her PreferenceSettings turn on for her prescription medication; a plan that violates it is rejected. A SoftConstraint is something to optimize, for example Ken’s “minimize external tool calls”, derived from his PreferenceSettings. Each SoftConstraint carries a user-given priority and is measured by a DecisionCriterion (Figure 6). `conflictsWith` links two soft constraints that push the same criterion in opposite directions, such as maximizing and minimizing tool use (alternate flow 3); the agent then asks the user to revise them. Constraints that only trade off, like Olivia’s strong evidence and few tool calls, are not conflicts: their priorities weight them. ExecutionConstraints come from planning: a Precondition must hold before a tool or skill runs, and an InputConstraint is a precondition on one input value.

Competency questions. CQs 1, 3, 4, 6 and 7 start by retrieving these constraints; alternate flow 3 uses `conflictsWith`.

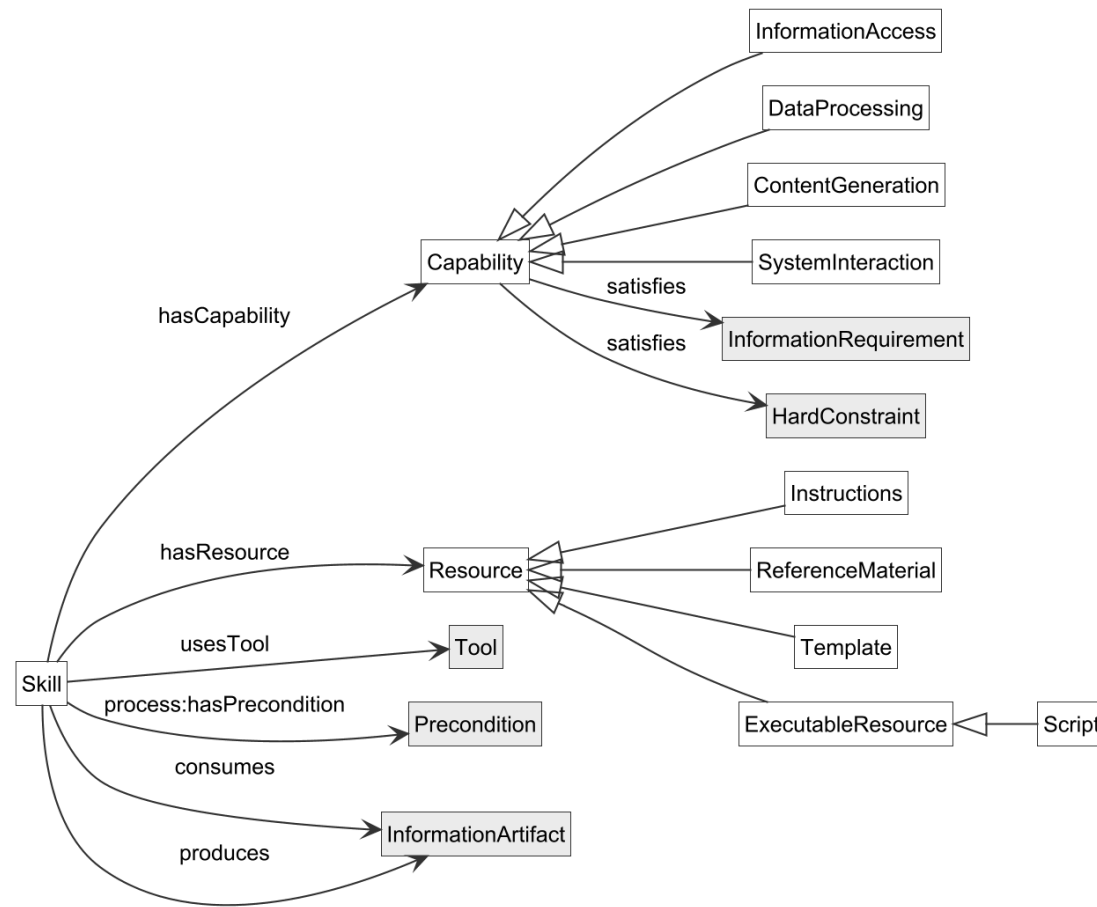


Figure 3. Capabilities, skills and skill resources

Shows. A Skill has Capabilities, bundles Resources (Instructions, ReferenceMaterial, Templates and ExecutableResources such as Scripts, as in the skill packages of SkillsBench [5]), uses Tools, has Preconditions, and consumes and produces InformationArtifacts. Capabilities come in four kinds and satisfy InformationRequirements and HardConstraints.

Modeling decisions. Skill is not a subclass of Capability, as the A4 term list had it. A skill is a packaged specification that has capabilities, and the same capability, such as screening for interactions, can come from a skill or directly from a tool; keeping Capability separate lets the reasoner compare the two. hasCapability widens AgentO’s property of the same name, whose domain is only Tool.

Competency questions. CQ 3 (Jessica): the safety-screening HardConstraint is satisfied by a capability of the Supplement Safety Screening Skill, so that skill is required. CQ 5 reads the same links from a plan’s tasks. CQ 8: the Evidence Verification and Supplement Safety Screening skills consume CandidateSupplement, which only the Personalized Supplement Recommendation Skill produces, so both depend on it.

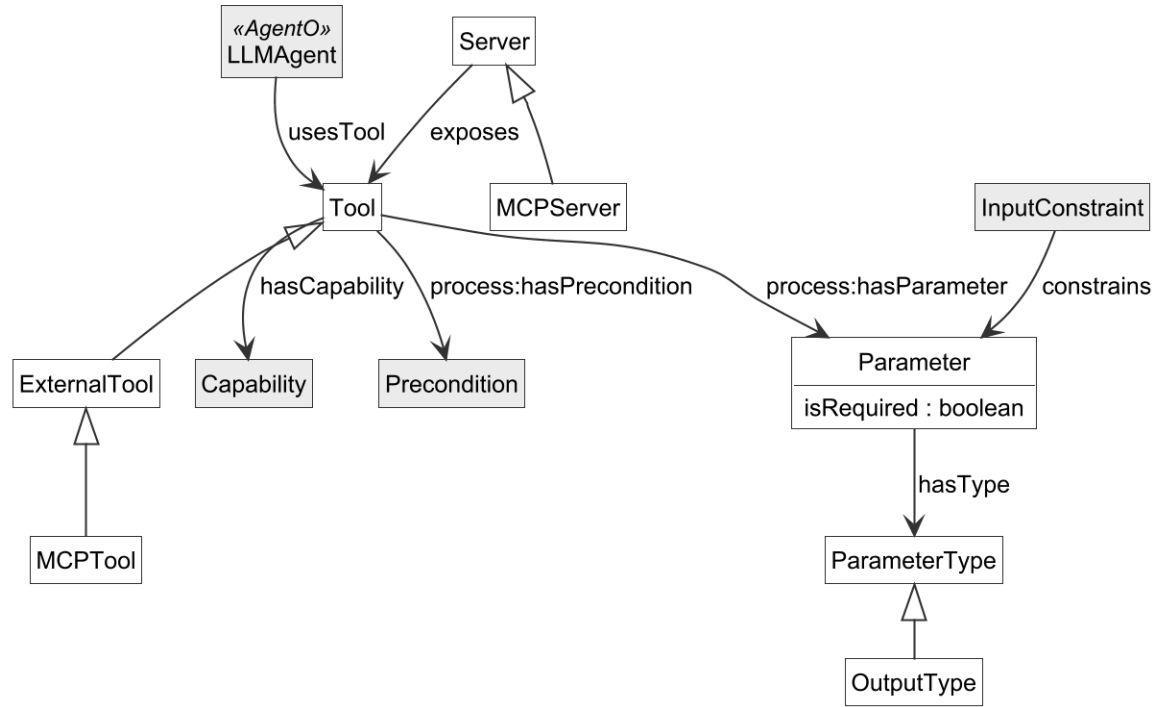


Figure 4. Tools, servers and tool interfaces

Shows. A Server exposes Tools. The PubMed, Supplement Advisor and OpenFDA servers are MCPServers [4] whose tools are MCPTools; iHerb-CLI’s commands are ExternalTools reached through a command line. A Tool’s interface is a set of Parameters, each with a ParameterType; OutputType types what a tool returns, isRequired marks required parameters, and an InputConstraint limits a parameter’s values. The LLMAgent uses tools.

Modeling decisions. Interface facts (parameters, types, preconditions) are kept apart from run-time facts (Figure 7), so the harness can check a plan before running it, as the third requirement in Section II of the use case asks. Reusing OWL-S [8] for parameters and preconditions also makes each tool an OWL-S process.

Competency questions. CQ 9 checks required parameters and preconditions before a call; alternate flow 2 lists the available tools with their capabilities.

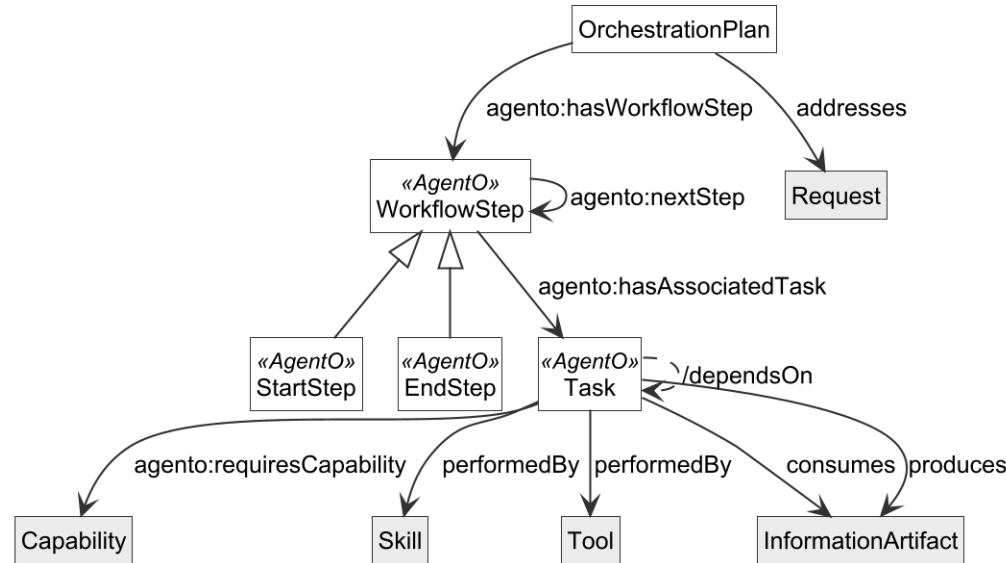


Figure 5. Plan structure and task dependencies

Shows. An *OrchestrationPlan* addresses one *Request* and has *WorkflowSteps*, ordered by *agento:nextStep* from a *StartStep* to an *EndStep*. Each step is associated with a *Task* that requires a *Capability*, is performed by a *Skill* or a *Tool*, and consumes and produces *InformationArtifacts*. Using *AgentO*'s *hasWorkflowStep* also makes every plan an *AgentO WorkflowPattern*, that is, a P-Plan plan.

Modeling decisions. Dependencies are derived, not entered by hand. Task B *dependsOn* Task A when B consumes an artifact that A produces; in OWL 2 this is the property chain $\text{consumes} \circ \text{produces}^{-1} \sqsubseteq \text{dependsOn}$ [14]. The KG therefore never stores a dependency that disagrees with the artifacts.

Competency questions. CQ 2 (Jessica): Screen interactions, Verify evidence and Get supplement products each consume the *CandidateSupplement* from Get candidate supplements, and none consumes another's output, so no *dependsOn* links them and they can run in parallel. CQ 4 (Dante) and CQ 8 read the order from the same derived links; CQ 9 finds the task that must run first.

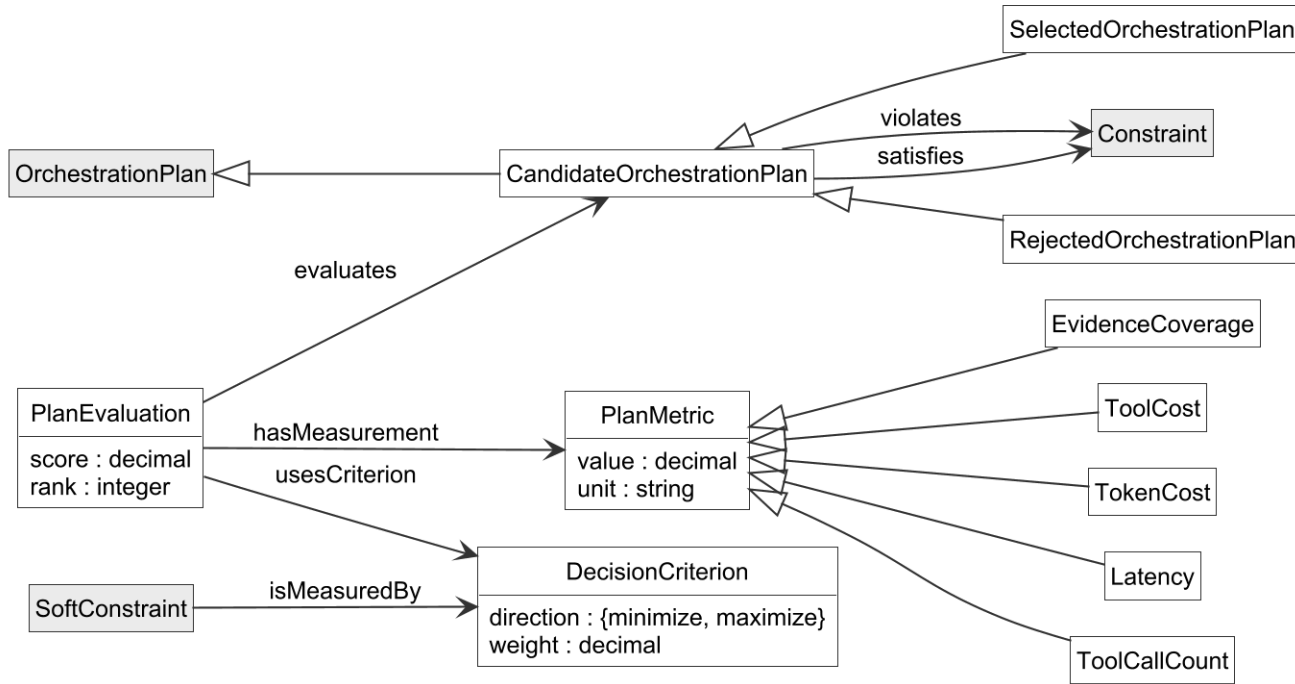


Figure 6. Plan evaluation and selection

Shows. Candidate plans for a request are compared on DecisionCriteria. A PlanEvaluation is one plan measured on one criterion: it links the CandidateOrchestrationPlan, the DecisionCriterion and the measured PlanMetric, and stores the score and rank, following the n-ary relation pattern [13]. A DecisionCriterion has a direction (minimize or maximize) and a weight taken from the priority of the SoftConstraint it measures. The metrics are ToolCallCount, EvidenceCoverage, ToolCost, TokenCost and Latency.

Modeling decisions. Selection runs in three steps, which is how conflicting criteria are handled. Hard constraints filter first: a candidate that violates one is rejected. The remaining candidates are ranked by their weighted criterion scores. Soft constraints that directly conflict (Figure 2) stop planning and go back to the user. The chosen plan is typed SelectedOrchestrationPlan and the others RejectedOrchestrationPlan, both stated explicitly: under the open-world assumption the KG cannot conclude that a plan was rejected just because it is not marked as selected.

Competency questions. CQ 6 (Olivia): Plans A and B both meet the evidence constraint; on the ToolCallCount criterion (minimize) Plan A measures 8 calls and Plan B 3, so B ranks first and is selected. CQ 7 follows Plan A's lower-ranked evaluation to its criterion and on to the soft constraint it measures, "use as few external tool calls as possible". CQ 1 (Ken) ranks by ToolCallCount and TokenCost; CQ 4 (Dante) by Latency.

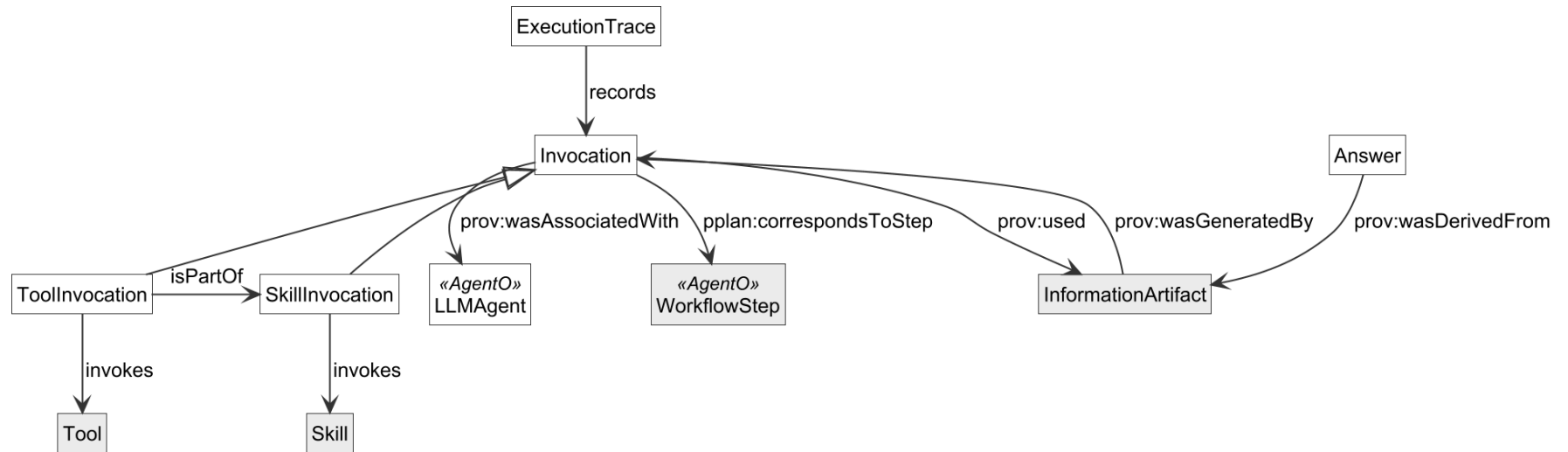


Figure 7. Execution and provenance

Shows. The run-time side of a plan, following P-Plan [12] and PROV-O [9]. Each Invocation corresponds to one WorkflowStep, is associated with the LLMAgent that ran it, and uses and generates InformationArtifacts. A ToolInvocation invokes a Tool and is part of the SkillInvocation that made the call; the three OpenFDA calls in Jessica’s plan, for example, are part of one Screen interactions invocation. The Answer is derived from the artifacts it summarizes, and an ExecutionTrace records the invocations, as basic flow step 11 requires.

Modeling decisions. The plan (Figure 5) and its execution are kept separate, so the KG can compare what was planned with what ran; reusing PROV-O and P-Plan instead of new terms keeps the record readable by existing provenance tools.

Competency questions. CQs 4 and 5 build their explanations from these records; CQ 9 checks which artifacts have been generated so far.

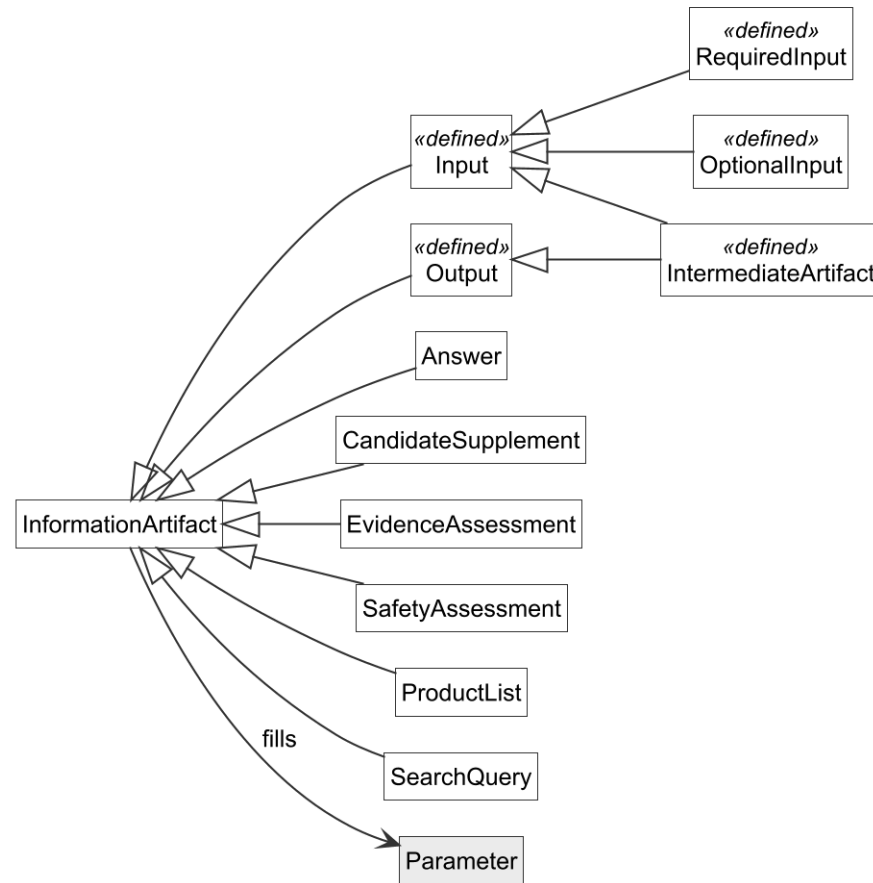


Figure 8. Information artifacts

Shows. The kinds of artifacts the use case names (SearchQuery, CandidateSupplement, EvidenceAssessment, SafetyAssessment, ProductList, Answer) and the roles an artifact can play. An artifact fills a tool Parameter when it is passed in a call.

Modeling decisions. Input, Output, RequiredInput, OptionalInput and IntermediateArtifact are roles, not kinds, following the party and role pattern from class. They are «defined» classes: an artifact is an Input because some task consumes it, and a RequiredInput is an input that fills a required parameter. The same CandidateSupplement is an Output of Get candidate supplements and an Input of Verify evidence, so the reasoner classifies it as an IntermediateArtifact.

Competency questions. CQ 9: before the Evidence Verification Skill runs, the harness checks that each of its required inputs (CandidateSupplement, SearchQuery) has been generated. The system writes these records itself, so this check can use a closed-world query, unlike checks on what external tools report.

How the model answers the competency questions

CQ	Scenario	Figures	Path through the model
1. What orchestration should the agent follow?	Ken	2, 3, 5, 6	Request → UserConstraints from his PreferenceSettings (SoftConstraints: few tool calls, low token use) → candidate plans whose tasks require capabilities that satisfy the InformationRequirements → PlanEvaluations on ToolCallCount and TokenCost → SelectedOrchestrationPlan
2. Which steps can run in parallel?	Jessica	5, 8	Tasks consume and produce artifacts → derived dependsOn → Screen interactions, Verify evidence and Get supplement products share only the CandidateSupplement input, so no dependsOn links them
3. What skills are required?	Jessica	2, 3	Request → expresses → InformationRequirement; her PreferenceSettings → safety-screening HardConstraint; each is satisfied by a Capability, and the Skills that have those capabilities are required
4. Why were capabilities invoked in this order?	Dante	2, 5, 6, 7	dependsOn gives the order; the safety-screening HardConstraint is satisfied by the screening capability; the SoftConstraint “minimize execution time” is measured by a Latency criterion that ranks the parallel plan first; invocation records supply the explanation
5. Why was each capability chosen?	Dante	3, 5, 7	Invocation → WorkflowStep → Task → requiresCapability → Capability → satisfies → InformationRequirement or HardConstraint
6. Why was plan B selected over plan A?	Olivia	6	PlanEvaluations of A and B on ToolCallCount (8 vs. 3) and EvidenceCoverage → weighted rank → B is SelectedOrchestrationPlan, A is RejectedOrchestrationPlan
7. Which constraints did a candidate plan violate?	Olivia	6	Plan A → its lower-ranked PlanEvaluation → usesCriterion → the SoftConstraint it measures (“minimize external tool calls”); violates records hard-constraint failures
8. What skills depend on other skills?	—	3, 5	Skills’ consumes and produces → CandidateSupplement is produced only by the recommendation skill → the evidence and screening skills depend on it
9. Are the prerequisites satisfied before a skill runs?	—	4, 7, 8	Required inputs and preconditions of the skill vs. artifacts generated so far (prov:wasGeneratedBy) → executable or blocked, plus the task that produces what is missing

References

Numbers follow Section IX of the use case; [12]–[14] are added to it in this revision, after [10] NIST and [11] IBM AI Atlas Nexus.

- [2] A. Ekelhart, K. Kurniawan, F. J. Ekaputra, and E. Kiesling, “AgentO: An ontology for modeling agentic AI systems,” in *The Semantic Web: 23rd European Semantic Web Conference, ESWC 2026, Proceedings, Part II, Lecture Notes in Computer Science*, vol. 16550, M. Acosta et al., Eds. Cham, Switzerland: Springer, 2026, pp. 298–320, doi: 10.1007/978-3-032-25159-6_16.
- [3] K. Kurniawan and F. J. Ekaputra, “AgentO: An ontology for modeling agentic AI systems,” ontology release v0.2, Zenodo, Jan. 22, 2026, doi: 10.5281/zenodo.18342625. [Online]. Available: <https://w3id.org/agentic-ai/onto>
- [4] Model Context Protocol Project, “Model Context Protocol specification,” revision 2026-07-28, LF Projects, LLC, Jul. 28, 2026. [Online]. Available: <https://modelcontextprotocol.io/specification/2026-07-28>
- [5] X. Li, Y. Liu, W. Chen, B. You, Z. Di, Y. He, et al., “SkillsBench: Benchmarking how well agent skills work across diverse tasks,” arXiv:2602.12670, Feb. 2026. [Online]. Available: <https://arxiv.org/abs/2602.12670>
- [8] D. Martin et al., “OWL-S: Semantic markup for web services,” W3C Member Submission, Nov. 22, 2004. [Online]. Available: <https://www.w3.org/submissions/OWL-S/>

- [9] T. Lebo, S. Sahoo, and D. McGuinness, Eds., “PROV-O: The PROV ontology,” W3C Recommendation, Apr. 30, 2013. [Online]. Available: <https://www.w3.org/TR/prov-o/>
- [12] D. Garijo and Y. Gil, “The P-Plan ontology,” version 1.3, Mar. 12, 2014. [Online]. Available: <http://purl.org/net/p-plan>
- [13] N. Noy and A. Rector, Eds., “Defining N-ary relations on the Semantic Web,” W3C Working Group Note, Apr. 12, 2006. [Online]. Available: <https://www.w3.org/TR/swbp-n-aryRelations/>
- [14] J. Bao, E. F. Kendall, D. L. McGuinness, and P. F. Patel-Schneider, Eds., “OWL 2 Web Ontology Language quick reference guide (second edition),” W3C Recommendation, Dec. 11, 2012. [Online]. Available: <https://www.w3.org/TR/owl2-quick-reference/>